

XRechnung-Hintergrunddienst — Entwicklerhandbuch

Übergabedokumentation für die Weiterentwicklung. Ergänzt die `README.md` (die den ursprünglichen Stand beschreibt) um die neu hinzugekommenen Funktionen und dient als Einstieg für Nachfolgeentwickler.

Repository: https://github.com/MuteStone/xrechnung_background_service · **Branch:** [exe](#) (hier läuft die aktuelle Entwicklung inkl. EXE-Build, PDF-Einbettung und KoSIT).

Inhalt 1. [Architektur & Datenfluss](#) 2. [Projektstruktur](#) 3. [Die Verarbeitungspipeline im Detail](#) 4. [Datenquellen & Datenbank](#) 5. [XRechnung-Erzeugung & PDF-Einbettung](#) 6. [Validierung: XSD und KoSIT](#) 7. [E-Mail-Versand](#) 8. [Konfiguration](#) 9. [Build & Packaging](#) 10. [Deployment](#) 11. [Tests](#) 12. [Bekannte Themen & Stolpersteine](#) 13. [Vorgeschlagene Erweiterungen](#) 14. [Schnellreferenz wichtiger Codestellen](#)

1. Architektur & Datenfluss

[illegible]

Es gibt **zwei Einstiegspunkte**, die dieselbe Pipeline fahren:

- **main.py** → `src/watcher/watcher.py` (`run_once` / `run_watch`) — der Dienst.
- **xrechnung_monitor.py** — GUI (PySide6), enthält eine **eigene Kopie** der Pipeline (`process_file`).

Achtung: **Wichtig:** Die Pipeline existiert an **zwei Stellen** (`watcher.py` und `xrechnung_monitor.py`). Änderungen an der Verarbeitungslogik müssen **in beiden** nachgezogen werden. Das ist historisch gewachsen und ein guter Kandidat für ein Refactoring (siehe [Abschnitt 13](#)).

2. Projektstruktur

```
xrechnung_background_service_clean/
■■ main.py                # Einstiegspunkt Dienst, CLI-Parsing
■■ xrechnung_monitor.py    # GUI-Monitor (PySide6) + eigene Pipeline (process_file)
■■ setup_wizard.py         # Einrichtungsassistent (PySide6), schreibt .env, kopiert Dateien
■■ build_all.bat           # Baut alle drei EXEs (Dienst → Monitor → Setup)
■■ XRechnung-Dienst.spec   # PyInstaller-Spec (bündelt xsd/)
■■ XRechnung-Monitor.spec  # PyInstaller-Spec (bündelt xsd/, PySide6)
■■ XRechnung-Setup.spec    # PyInstaller-Spec (bündelt Dienst+Monitor+kosit/)
■■ .env / .env.example     # Konfiguration
■■
■■ src/
```

```

■ ■ ■ database/db.py          # MySQL-Anbindung, get_invoice_full(), Export-Jobs
■ ■ ■ watcher/watcher.py      # Pipeline: run_once(), run_watch(), _process_single_pdf()
■ ■ ■ transmitter/transmitter.py # SMTP-Versand + Protokoll-Mail (send_report)
■ ■ ■ utils/
■ ■ ■ ■ ■ config.py           # .env laden → Konfig-Dictionary
■ ■ ■ ■ ■ logger.py           # Rotating-File- + Konsolen-Logger, Lauf-Log
■ ■ ■ xrechnung/
■ ■ ■ ■ ■ pdf_reader.py       # Rechnungsnummer/Adresse/Datum aus Dateiname & PDF-Inhalt
■ ■ ■ ■ ■ pdf_seller_reader.py # Verkäuferdaten aus PDF (Setup-Assistent)
■ ■ ■ ■ ■ json_reader.py      # ePost-JSON (companion) auslesen
■ ■ ■ ■ ■ generator.py        # CII-XML-Generierung + PDF-Einbettung + Größenprüfung
■ ■ ■ ■ ■ validator.py        # XSD-Validierung (nicht-blockierend)
■ ■ ■ ■ ■ kosit_validator.py   # KoSIT-Validierung (Java-Subprozess, blockierend)
■ ■ ■ ■ ■ xsd/                 # CII-D16B-Schemadateien (gebündelt)
■ ■ ■
■ ■ ■ tools/
■ ■ ■ ■ ■ fetch_kosit.py      # Build-Werkzeug: beschafft kosit/ (JRE+Validator+Szenario)
■ ■ ■
■ ■ ■ kosit/                  # (nicht im Git) Java-Laufzeit + Validator + Szenario
■ ■ ■ docs/                   # dieses Handbuch + Anwenderhandbuch
■ ■ ■ tests/                  # pytest
■ ■ ■ logs/ output/ processed/ error/

```

3. Die Verarbeitungspipeline im Detail

Referenz: `xrechnung_monitor.py` → `process_file()` bzw. `watcher.py` → `_process_single_pdf()`. Schritte:

1. **Rechnungsnummer bestimmen** — aus dem Dateinamen (`pdf_reader.extract_invoice_number`, Muster `Rechnung_YYYYMMDD-NNN.pdf`), sonst aus dem PDF-Inhalt, bei JSON aus der JSON.
2. **Rechnungsdaten laden** — Reihenfolge/Zusammenführung: - JSON (companion oder standalone), falls vorhanden, - Datenbank (`get_invoice_full` per Nummer bzw. `get_invoice_by_dokumenteid`), - PDF-Extraktion (Empfängeradresse, Lizenznehmer-ID, Leistungsbeginn), - Verkäufer-Fallback aus DB-Profil bzw. `.env` (`_apply_seller_fallback`).
3. **XML erzeugen** — `generator.generate(invoice_data, out_dir, pdf_path=source_pdf)`. Ausgabe: `PROCESSED_FOLDER/<pdf-stem>/<rechnungsnr>.xml`. Die Quell-PDF wird eingebettet (nur wenn die Quelle eine PDF ist; JSON-Quellen: `pdf_path=None`).
4. **XSD-Prüfung** — `validator.validate(xml_path)`. **Nicht-blockierend** (s. u.).
5. **KoSIT-Prüfung** — nur wenn `KOSIT_VALIDATION=true`. **Blockierend & fail-closed**: bei Ablehnung oder fehlenden Werkzeugen → `_move_to_error`, kein Versand. Läuft auch im Dry-Run (nur der Versand wird übersprungen).
6. **Versand** — `transmitter.transmit(xml_path, config)` (kein `pdf_path` → nur XML).
7. **Verschieben** — Erfolg: `processed/`, Fehler: `error/` (mit Begründung im Log).

4. Datenquellen & Datenbank

Reihenfolge der Datenermittlung

Käuferdaten: JSON → DB → PDF → Fallback. Verkäuferdaten: DB-Profil → `.env`. Leitweg-ID: JSON → DB (`tb_kundenzusatz`) → Lookup über Lizenznehmer-ID aus PDF.

Companion-JSON (PDF+JSON-Paare)

Eine JSON mit derselben Rechnungsnummer wie eine PDF wird **immer** als Companion erkannt (`_collect_files`) — unabhängig von `SCAN_JSON`. Das bedeutet: Die JSON wird **mit der PDF verschoben** (nach `processed/error/`) **und für die Datenanreicherung mitgelesen** (ePost: `dokumenteid`, Käuferadresse). `SCAN_JSON` steuert nur noch, ob **alleinstehende** JSONs (ohne passende PDF) eigenständig verarbeitet werden.

Datenbank (`src/database/db.py`)

Die SQL-Abfragen sind auf eine **konkrete Kundenverwaltungs-DB** zugeschnitten. Bei Einsatz gegen eine andere DB müssen Tabellen-/Spaltennamen **und** die Abfragen angepasst werden. Relevante Tabellen:

Zweck	Tabelle	Funktion
Rechnungskopf	<code>tb_dokumente</code>	<code>get_invoice_full</code> , <code>get_invoice_by_dokumenteid</code>
Positionen	<code>tb_dokpositionen</code>	dito
Kundenstamm	<code>tb_kunden</code>	<code>get_invoice_full</code>
Kundenzusatz (Leitweg-ID, Rechnungs-Mail)	<code>tb_kundenzusatz</code>	<code>_fetch_leitweg_id</code> , <code>_fetch_rechnung_mail</code>
Rechnungsadresse	<code>tb_kunden_adressen</code> (<code>adressenart=2</code>)	<code>_fetch_billing_address</code>
Bundesländer	<code>tb_bundeslaender</code>	<code>get_invoice_full</code>
Übertragungsprotokoll	<code>export_jobs</code>	<code>get_export_job_by_pdf_path</code> , <code>update_export_job_status</code>

Anzupassende Konstanten oben in `db.py`:

<code>_PROPERTY_ROUTING_ID</code>	<code>= 29</code>	<code># eigenschaft-ID der Leitweg-ID in tb_kundenzusatz</code>
<code>_PROPERTY_INVOICE_EMAIL</code>	<code>= 27</code>	<code># eigenschaft-ID der Rechnungs-E-Mail</code>
<code>_TYPE_INVOICE</code>	<code>= 3</code>	<code># typ-Wert, der eine Rechnung kennzeichnet</code>

Verbindung: PyMySQL, `DictCursor`, `autocommit=True`, `utf8mb4`, `connect_timeout=10` (verhindert endloses Hängen bei nicht erreichbarem DB-Host).

5. XRechnung-Erzeugung & PDF-Einbettung

`src/xrechnung/generator.py` baut die CII-XML mit `lxml`.

- **Format:** EN 16931, CII D16B, XRechnung 3.0 (Guideline-ID gesetzt).
- **Leitweg-ID-Normalisierung:** Vor dem Schreiben werden **alle Leerzeichen/ Whitespace** aus der Leitweg-ID entfernt (`re.sub(r"\s", "", ...)`), da das OZG-RE-Portal eine ID mit Leerzeichen nicht erkennt. Wirkt zentral für `BuyerReference` und die 0204-Empfängerkennung — deckt alle Quellen (DB/JSON/PDF) ab.
- **Wichtige BT-Felder:** `BuyerReference` (Leitweg-ID), `Seller/Buyer-Party` inkl. elektronischer Adresse, `DefinedTradeContact` (Name/Telefon/E-Mail — **BR-DE-5/6/7**), `Liefer-/Leistungsdatum` (BT-72), `Zahlungsmittel` (IBAN/BIC), `Steuergruppen`, `Summenblock`.
- **PDF-Einbettung (BT-125):** `_embed_pdf_attachment()` hängt an das `ApplicableHeaderTradeAgreement` ein `ram:AdditionalReferencedDocument` mit `ram:AttachmentBinaryObject` (Base64,

`mimeCode="application/pdf", TypeCode 916`). Das ist der einzige vom OZG-RE unterstützte Weg, die PDF an den Empfänger durchzureichen — ein separater E-Mail-Anhang wird verworfen.

- **Größenprüfung:** Nach dem Schreiben wird die XML-Dateigröße gegen `MAX_XML_BYTES` (10 MB, OZG-RE-E-Mail-Limit) geprüft. Überschreitung → `XRechnungError`.
- **Abbruchverhalten:** `generate()` gibt bei „normalen“ Fehlern `None` zurück; bei harten Abbruchbedingungen (PDF fehlt/unlesbar, XML zu groß) wirft es `XRechnungError`, das die Pipeline als Fehlermeldung in den `error/-`Report übernimmt.

6. Validierung: XSD und KoSIT

6.1 XSD (`validator.py`) — nicht-blockierend

Prüft nur gegen das CII-XSD. **Achtung:** Es blockiert **nur bei XML-Syntaxfehlern**; echte Schemaverstöße werden lediglich als Warnung geloggt und **durchgelassen** (`return True`). Die inhaltliche Prüfung übernimmt KoSIT.

6.2 KoSIT (`kosit_validator.py`) — blockierend, fail-closed

Ruft den offiziellen **KoSIT-Validator** (Java) mit dem XRechnung-Szenario auf und wertet den VARL-Report aus. Prüft die normativen Geschäftsregeln (BR-DE-*, EN 16931).

Layout `kosit/` (neben der EXE bzw. im Projektstamm):

```
kosit/  
jre/                # getrimmte Java-Laufzeit (jre/bin/java.exe)  
validator/<tool>.jar # validationtool-*-standalone.jar  
scenario/scenarios.xml # XRechnung-Konfiguration inkl. resources/
```

Pfadauflösung: frozen → `Path(sys.executable).parent / "kosit"`; dev → Projektstamm.

Aufruf: `java -jar <jar> -r <scenario-dir> -s scenarios.xml -o <tmp> <xml>`. Erfolg/Ablehnung wird **versions-robust** aus dem Report gelesen (Suche per local-name nach `accept/reject`); Unklarheit/fehlender Report → ungünstig (fail-closed).

Achtung: Windows-Stolperstein (beheben, nicht entfernen): Der Validator prüft `System.in.available()`. Ohne stdin bzw. mit `NUL` wirft Java `IOException: Unzulässige Funktion` und erzeugt **keinen** Report. Lösung im Code: `subprocess.run(..., input="")` — eine echte, leere Pipe. Siehe Kommentar in `validate_kosit()`.

Fehlt `kosit/` und `KOSIT_VALIDATION=true`: `available=False` → Pipeline bricht ab (bewusst — kein ungeprüfter Versand). Für die reine Python-Entwicklung ohne Artefakte in der lokalen `.env` `KOSIT_VALIDATION=false` setzen.

6.3 Artefakte beschaffen/aktualisieren (`tools/fetch_kosit.py`)

Build-Werkzeug (läuft **nicht** in der EXE). Es lädt via GitHub-API das neueste Validator-JAR (`itplr-kosit/validator`) und die XRechnung-Konfiguration (`itplr-kosit/validator-configuration-xrechnung`), entpackt das Szenario und baut per `jlink` eine getrimmte JRE (Modulsatz `java.se`).

```
python tools/fetch_kosit.py          # neueste Releases automatisch
python tools/fetch_kosit.py --jdk "C:/Program Files/Java/jdk-17"
python tools/fetch_kosit.py --skip-jre # nur Validator+Szenario (System-Java)
```

Voraussetzung für den JRE-Bau: ein JDK 11+ mit `bin/jlink` und `jmods/`.

Versions-Update: Bei neuer XRechnung-Version `fetch_kosit.py` erneut ausführen (holt automatisch das neueste Release), dann EXEs neu bauen. Der Report-Parser ist elementnamen-robust und sollte ohne Codeänderung mitziehen.

7. E-Mail-Versand

`src/transmitter/transmitter.py`:

- `transmit(xml_path, config)` — versendet die XRechnung. **Die Pipeline übergibt bewusst keinen pdf_path**, d. h. die E-Mail enthält **nur die XML** (die PDF steckt darin). Das entspricht der OZG-RE-Regel „genau eine XRechnungs-Datei pro E-Mail, keine weiteren Anhänge“.
- `send_report(...)` — optionale Protokoll-Mail nach dem Lauf (an `REPORT_EMAIL`).

Achtung: **Verschlüsselung:** `_send()` nutzt aktuell **einfaches SMTP ohne STARTTLS** (IPv4 erzwungen, kein `starttls()`) — passend für einen internen Mail-Relay. Die `README.md` spricht abweichend von STARTTLS/Port 587; das ist Dokumentations-Drift. Für einen TLS-pflichtigen Anbieter muss `_send()` angepasst werden (`smtplib.SMTP_SSL` bzw. `starttls()`).

8. Konfiguration

`src/utils/config.py` lädt die `.env` (neben der EXE) in ein Dictionary mit Defaults. Neuen Schalter hinzufügen bedeutet an **drei** Stellen ergänzen: `config.py` (Lesen + Default), `.env.example` (Doku) und `setup_wizard.py` (`write_env`, ggf. GUI-Feld). Beispiel: `KOSIT_VALIDATION`.

9. Build & Packaging

PyInstaller. `build_all.bat` ist das **einzige Build-Skript** und baut in der zwingenden Reihenfolge **Dienst** → **Monitor** → **Setup** (Setup bündelt die beiden fertigen EXEs + `kosit/`). Ein „nur Setup bauen“ gibt es nicht — die Dienst-/ Monitor-EXEs müssen vorher existieren. `build_all.bat` warnt, wenn `kosit/` fehlt.

```
.\build_all.bat          # in PowerShell den führenden .\ nicht vergessen
```

EXE	Spec	Bündelt	Konsole
Dienst	<code>XRechnung-Dienst.spec</code>	<code>xsd/</code>	ja
Monitor	<code>XRechnung-Monitor.spec</code>	<code>xsd/</code> , PySide6	nein
Setup	<code>XRechnung-Setup.spec</code>	Dienst.exe, Monitor.exe, <code>kosit/</code> (falls vorhanden)	nein, <code>uac_admin</code>

KoSIT-Bundling-Konzept: `kosit/` wird **nur in die Setup.exe** als `datas` aufgenommen (siehe bedingtes `_kosit_datas` in der Spec). Der Setup-Wizard kopiert `kosit/` beim „Übernehmen“ **neben** die installierten

EXEs. Es wird bewusst **nicht** in Dienst/Monitor gebacken — sonst würde die große JRE bei jedem Start ins Temp entpackt. Zur Laufzeit lesen Dienst/Monitor `kosit/` neben ihrer eigenen EXE.

Voraussetzung für einen vollständigen Setup-Build: vorher `tools/fetch_kosit.py` ausführen, damit `kosit/` existiert. Ohne `kosit/` baut die Setup.exe zwar, enthält aber keine Validierungswerkzeuge.

10. Deployment

Produktivumgebung: `P:\Kundenverwaltung\XRechnung Automatisierung`. Dort müssen neben den EXEs auch `kosit\` und die produktive `.env` liegen (die EXEs lösen beides relativ zu sich auf).

Build nur lokal: Auf einem Netzlaufwerk (P:Y:) wird **nicht** gebaut — dort fehlt die `.venv`, `build_all.bat` bricht ab. Erst eine lokale Arbeitskopie anlegen (siehe `CLAUDE.md`), lokal bauen, dann die Artefakte kopieren.

Update einer bestehenden Installation (Normalfall)

Nach `build_all.bat` die beiden neu gebauten EXEs nach P: kopieren: `dist\XRechnung-Dienst.exe` und `dist\XRechnung-Monitor.exe`. `kosit\` und `.env` bleiben dort unangetastet (nur bei neuem KoSIT-Szenario auch `kosit\` mitkopieren).

Erstinstallation über das Setup

- `XRechnung-Setup.exe` **lokal**, mit **Administratorrechten** (UAC) ausführen — **nicht** auf dem Netzlaufwerk.
- Der Wizard legt EXEs + `kosit\` + `.env` in den gewählten Ordner; diesen dann nach P: kopieren.

Achtung: Produktiv-.env nicht überschreiben

Das Setup **erkennt keine bestehende Installation** und startet mit **leeren Feldern**. Beim Update daher **nur die EXEs** kopieren, **nicht** die `.env` — sonst wird die funktionierende Produktiv-Konfiguration mit leeren Werten überschrieben. Im Zweifel die Produktiv-`.env` vorher sichern.

Weiteres

- **Auslöser:** Task Scheduler (Programm = EXE, „Ausführen in“ = Ordner der EXE).
- **Portable Alternative (nur Datenzugriff):** Ordner mit EXE + `kosit\` + angepasster `.env` läuft überall; auf dem Ziel `.env`-Pfade anpassen, DB/SMTP- Erreichbarkeit prüfen.

11. Tests

```
.venv\Scripts\python.exe -m pytest tests/ -q
```

Bekannter Fehlschlag: `tests/test_generator.py::TestExtractInvoiceNumber::test_kleinschreibung_kein_match` schlägt fehl. Ursache liegt in `pdf_reader` (Groß-/Kleinschreibung im Dateinamen-Muster) und ist **unabhängig** von den XRechnung-/KoSIT-Änderungen — Bestand seit vor dieser Arbeit. Alle übrigen Tests laufen grün. Entweder das Regex-Muster case-sensitiv machen oder den Test an das gewünschte Verhalten anpassen.

Der KoSIT-Report-Parser lässt sich ohne Java gegen synthetische VARL-Reports testen (accept/reject/leer). Das Zusammenspiel mit dem echten Validator wurde manuell gegen eine Produktivrechnung verifiziert (bestanden).

12. Bekannte Themen & Stolpersteine

- **Doppelte Pipeline** in `watcher.py` und `xrechnung_monitor.py` — Änderungen immer in beiden nachziehen (Refactoring-Kandidat).
- **BR-DE-6: SELLER_PHONE** ist Pflicht, sonst lehnt KoSIT jede Rechnung ab. Auch Verkäufer-Name/E-Mail (BR-DE-5/7) müssen vorhanden sein.
- **KoSIT-Windows-stdin-Quirk** (Abschnitt 6.2) — `input=""` nicht entfernen.
- **10-MB-Grenze:** Große PDFs führen zum Abbruch. Für >15 MB sieht OZG-RE „Große Anlagen“ per Verweis (BT-124) statt Einbettung vor — hier nicht implementiert.
- **SMTP ohne TLS** (Abschnitt 7) — bei externem Anbieter anzupassen.
- **PowerShell:** Skripte/EXEs im aktuellen Ordner mit `.\` starten (`.\build_all.bat`).
- **KoSIT-Version** ist gepinnt (in `kosit/`); bei XRechnung-Updates `fetch_kosit.py` erneut laufen lassen und neu bauen.

Robustheits-Härtungen (nach der Erst-Doku ergänzt)

- **Logger toleriert Ordner-Pfad:** Zeigt `LOG_FILE` (fehlkonfiguriert) auf ein bestehendes Verzeichnis, hängt `_resolve_log_path` automatisch `xrechnung_dienst.log` an. Zuvor stürzte der `FileHandler` mit `PermissionError` ab und der Dienst starb **vor** der Verarbeitung (Symptom: „Fenster offen, nichts passiert“). Siehe [logger.py](#).
- **DB-Verbindung mit `connect_timeout=10`:** verhindert endloses Hängen bei nicht erreichbarem DB-Host. Siehe [db.py](#).
- **Leitweg-ID-Whitespace wird entfernt** (siehe Abschnitt 5) — OZG-RE erkennt sonst die ID nicht.

13. Vorgeschlagene Erweiterungen

- **Pipeline zusammenführen:** die geteilte Logik aus `watcher.py` und `xrechnung_monitor.py` in ein gemeinsames Modul ziehen.
- **Leitweg-ID-Prüfziffer offline prüfen** (fängt Tippfehler vor dem Versand). Der Algorithmus ist ISO 7064 MOD 97-10 (wie IBAN); verifiziert gegen eine echte ID: `python # Grob- + Feinadressierung ohne Prüfziffer, Buchstaben → Position (A=10..Z=35) # Prüfziffer = 98 - ((zahl * 100) mod 97); Validierung: int(zahl+prüfziffer) % 97 == 1`
- **Peppol-Teilnehmer-Lookup** (Empfängerkennung `0204:<Leitweg-ID>`) als *Warnung*, nicht als harter Abbruch (nur aussagekräftig, wenn der Empfänger über Peppol empfängt).
- **KoSIT-Daemon-Modus** statt JVM-Start pro Rechnung (Performance bei großen Läufen).

14. Schnellreferenz wichtiger Codestellen

Thema	Datei	Symbol
Einstieg Dienst / CLI	<code>main.py</code>	<code>main, parse_args</code>

Pipeline (Dienst)	<code>src/watcher/watcher.py</code>	<code>run_once, run_watch,</code> <code>_process_single_pdf</code>
Pipeline (GUI)	<code>xrechnung_monitor.py</code>	<code>process_file</code>
XML + PDF-Einbettung	<code>src/xrechnung/generator.py</code>	<code>generate,</code> <code>_embed_pdf_attachment,</code> <code>MAX_XML_BYTES, XRechnungError</code>
XSD-Prüfung	<code>src/xrechnung/validator.py</code>	<code>validate</code>
KoSIT-Prüfung	<code>src/xrechnung/kosit_validator.py</code>	<code>validate_kosit, _parse_report,</code> <code>kosit_available</code>
Versand	<code>src/transmitter/transmitter.py</code>	<code>transmit, _send, send_report</code>
Datenbank	<code>src/database/db.py</code>	<code>get_invoice_full,</code> <code>get_invoice_by_dokumenteid,</code> Konstanten oben
Konfiguration	<code>src/utils/config.py</code>	<code>load_config</code>
Artefakt-Beschaffung	<code>tools/fetch_kosit.py</code>	<code>main, build_jre,</code> <code>fetch_scenario</code>
Bundling	<code>XRechnung-Setup.spec,</code> <code>setup_wizard.py</code>	<code>_kosit_datas,</code> <code>PageFinish._apply</code>